

1. ❏ Introduction to Bash CLI & Error Handling

❏ What is Bash?

Bash (Bourne Again SHell) is the default command-line interface and scripting shell for most Linux distributions.

❏ Error Handling in Bash

Technique	Purpose
exit	Exit script with status (0 = success)
set -e	Exit immediately if a command fails
set -u	Exit if an unset variable is used
set -o pipefail	Return failure if any command in pipeline fails
\$?	Returns exit status of last executed command

Example:

bash

Copy code

```
set -e cp file.txt /tmp/ || echo "Copy failed"
```

2. ☒ Debugging & Redirection of Scripts

☒ Debugging Methods

Option	Description
bash -x script.sh	Prints each command before execution
set -x	Debug mode for parts of the script
trap	Trap signals and print messages for debugging

☒ Redirection Symbols

Symbol	Purpose
>	Redirect stdout (overwrite)
>>	Append stdout
2>	Redirect stderr
2>&1	Merge stderr into stdout
<	Input redirection

Example:

bash

Copy code

```
./myscript.sh > out.log 2>&1
```

3. ☒ Control Structures & Loops

☒ Looping Constructs

for loop

bash

Copy code

```
for i in {1..5}; do echo "Item $i"; done
```

while loop

bash

Copy code

```
count=0 while [ $count -lt 5 ]; do echo $count ((count++)) done
```

until loop

bash

Copy code

```
until [ "$var" = "done" ]; do echo "Not done yet" sleep 1 done
```

4. ☒ Variables & Strings

☒ Variables

bash

Copy code

name="Linux" echo "Welcome \$name"

Feature	Example
Declare integer	declare -i num=10
Read-only	readonly path="/usr/bin"
Export to env	export PATH="\$PATH:/new/path"

☒ String Operations

bash

Copy code

```
str="BashScripting" echo "${str:0:4}"    # Substring echo "${#str}"          # Length echo "${str/Scripting/Programming}" # Replace
```

5. ☒ Conditional Statements

☒ if-else Syntax

bash

Copy code

```
if [ condition ]; then # Commands elif [ another_condition ]; then # Commands else # Commands fi
```

☒ Common Operators

Type	Operators
Integer	-eq, -ne, -lt, -le, -gt, -ge
String	=, !=, -z, -n
File	-f, -d, -r, -w, -x, -e

6. ☒ Regular Expressions in Bash

☒ What is a Regex?

A **regular expression** is a pattern that describes a set of strings.

☒ Common Uses in Bash:

- Pattern matching with `[[ =~ ]]`

- Grep, sed, awk integration

Example:

bash

Copy code

```
if [[ "$email" =~ ^[a-zA-Z0-9._%+-]+@[a-z]+\.[a-z]{2,}$ ]]; then echo "Valid email" else echo "Invalid email" fi
```

📌 Important Metacharacters:

Character	Meaning
.	Any character
*	Zero or more
+	One or more
?	Optional
^	Start of string

\$	End of string
[abc]	Match a, b, or c